Alinhamento Técnico: Site Oficial do Game & API REST Backend (.NET 9)

Módulos Core, Sistema, Acesso (5 Perfis), Catálogo de Endpoints de Produção e Desacoplamento do Kanban

Projeto: Fowlgen Wars (G5B Studios)
Data: 20 de Outubro de 2026
Backend / API (.NET 9): Marcos (Arquiteto)
Site Oficial Público: Junior (Desenvolvedor Web)

1. Diretriz de Engenharia & Decisão Estratégica do Kanban

• **Desacoplamento do Kanban:** O projeto kanban/ será desacoplado futuramente para se consolidar como uma ferramenta multi-projeto independente para visualização de vários produtos. Por enquanto, mantemos o Kanban como ferramenta interna estável de CLI (npm run dashboard).
• **Foco da API REST:** Toda a arquitetura do backend em .NET 9 é direcionada para suportar a operação do **Site Oficial Público** e do jogo, com catálogo completo de rotas, RBAC e módulo de apoiadores.

2. Separação de Responsabilidades por Frente

Frente / Módulo	Escopo Principal	Público-Alvo	Responsável
Site Oficial Público (app/)	Vitrine gamer, download do APK, container Unity WebGL, mural público dinâmico de apoiadores (GET /api/supporters) e página Apoie-nos.	Comunidade, visitantes e jogadores (Player)	**Junior**
API REST Backend (api/src/backend/)	Arquitetura .NET 9 / Onix.Framework: Módulos Core, Sistema e Acesso (5 Perfis RBAC), CRUD paginado, Supporters e healthcheck.	Consumida pelo Site Público e pelo Game	**Marcos**
Kanban Interno (kanban/)	Sincronização OpenSpec e dashboard mantidos como ferramenta interna (projeto multi-projetos futuro desacoplado).	Equipe técnica interna G5B Studios	**Marcos**

3. Catálogo Completo de Endpoints da API REST (.NET 9 / Onix.Framework)

Catálogo unificado de produção com todos os endpoints implementados na API para atender o ecossistema:

Método & Rota	Controller	Acesso RBAC	Payload / Parâmetros	Descrição & Finalidade
GET /api/users	Users	Staff, Admin	pageIndex, pageSize, status, search	Listagem paginada de usuários com filtros e busca
GET /api/users/{id}	Users	Owner, Admin	Path: id (Guid)	Detalhes cadastrais do perfil do usuário por ID
POST /api/users	Users	Público	{ email, nickname, password }	Cadastro inicial de usuário/jogador no portal web
PUT /api/users/{id}	Users	Owner, Admin	{ nickname, avatarUrl }	Atualização de dados cadastrais (nickname, avatar)
PATCH /api/users/{id}/status	Users	Mod, Admin	{ status, motivo }	Alterar status da conta: Ativo, Inativo, Bloqueado
DELETE /api/users/{id}	Users	Admin	Path: id (Guid)	Exclusão lógica definitiva de usuário
GET /api/users/{id}/wallet	Users	Server, Admin	Path: id (Guid)	Consulta de carteira associada para servidor FishNet
POST /api/auth/login-web3	Auth	Público	{ walletAddress, signature, msg }	Login Web3 Solana (assinatura) e emissão de JWT
POST /api/auth/login-institucional	Auth	Público	{ email, password }	Login corporativo com senha e Salt (Admin/Staff/Mod)
POST /api/auth/refresh-token	Auth	Autenticado	{ refreshToken }	Renovação de token JWT de sessão expirado
GET /api/perfis	Perfis	Staff, Admin	pageIndex, pageSize	Listagem paginada dos 5 perfis oficiais
GET /api/perfis/{id}	Perfis	Staff, Admin	Path: id (Guid)	Detalhes do perfil com suas permissões vinculadas
POST /api/perfis	Perfis	Admin	{ nome, descricao }	Criação de novo perfil de acesso customizado
PUT /api/perfis/{id}	Perfis	Admin	{ nome, descricao }	Atualização de nome e descrição do perfil
PATCH /api/perfis/{id}/status	Perfis	Admin	{ ativo: boolean }	Ativação ou desativação de perfil de acesso
POST /api/perfis/{id}/permissoes	Perfis	Admin	{ permissaoIds: Guid[] }	Vincular conjunto de permissões técnicas ao perfil
GET /api/permissoes	Permissões	Staff, Admin	pageIndex, pageSize, modulo	Catálogo paginado de permissões por módulo
GET /api/permissoes/{id}	Permissões	Staff, Admin	Path: id (Guid)	Detalhes de uma permissão técnica por ID
POST /api/permissoes	Permissões	Admin	{ codigo, nome, modulo, desc }	Cadastro de nova permissão técnica
PUT /api/permissoes/{id}	Permissões	Admin	{ nome, modulo, descricao }	Atualização cadastral da permissão técnica
PATCH /api/permissoes/{id}/status	Permissões	Admin	{ ativo: boolean }	Ativação ou inativação de permissão técnica
GET /api/supporters	Supporters	Público	pageIndex, pageSize	Mural público paginado (apenas status Aprovado)

GET /api/supporters/admin	Supporters	Staff, Mod, Admin	pageIndex, pageSize, status	Fila administrativa com auditoria completa de apoio
GET /api/supporters/{id}	Supporters	Público / Admin	Path: id (Guid)	Detalhes do registro de apoio comunitário
POST /api/supporters	Supporters	Público / User	{ nickname, wallet, tier, tx, msg }	Registro de apoio comunitário e tier selecionado
PUT /api/supporters/{id}	Supporters	Owner, Admin	{ nickname, mensagem }	Atualização cadastral do registro de apoio
PATCH /api/supporters/{id}/status	Supporters	Mod, Admin	{ status: Pendente/Aprovado/Oculto }	Moderação de status: Pendente, Aprovado, Oculto
DELETE /api/supporters/{id}	Supporters	Admin	Path: id (Guid)	Remoção física/lógica de apoio irregular/fraudulento
GET /api/health	Health	Público	N/A	Healthcheck e verificação de integridade do PostgreSQL

4. Matriz de Tarefas Sincronizadas no Kanban Studio (OpenSpec)

Trilha 1: Site Oficial Público do Game (app/) — Responsável: @Junior

• [ ] **2.1:** Página de vitrine do MVP com botão de download do APK Android.

• [ ] **2.2:** Preparação do container de apresentação para embutir o jogo Unity WebGL.

• [ ] **2.6:** Refatorar sistema de cadastro/nickname: trocar prompt local por formulário/modal conectado a POST /api/users.

• [ ] **2.7:** Refatorar autenticação Web3: conectar assinatura de carteira a POST /api/auth/login-web3 e salvar JWT.

• [ ] **2.8:** Refatorar modelo do jogador no front-end para campos da API (id_usuario, nickname_jogo, email, wallet_address).

• [ ] **2.9:** Refatorar e modernizar estilo visual do site (CSS/Tailwind, botões 3D táteis, tipografia e responsividade mobile).

• [ ] **2.10:** Criar interface 'Apoie-nos' com apresentação de Tiers e formulário de contribuição.

• [ ] **2.11:** Criar página/mural 'Apoiadores' com feed dinâmico consumindo a API (GET /api/supporters).

• [ ] **2.12:** Atualizar navegação na Navbar e Footer com links para 'Apoiadores' e 'Apoie-nos'.

• [ ] **2.13:** Implementar paginação no client HTTP do site para consumir as listagens paginadas.

• [ ] **2.14:** Tratar e exibir badges visuais de status da conta do jogador (Ativo, Inativo, Bloqueado).

Trilha 2: Arquitetura de Produção na API Backend para o Site (api/) — Responsável: @Marcos

• [ ] **3.1:** Implementação no módulo Core e Acesso dos Value Objects (Senha e Salt com hashing seguro).

• [ ] **3.2:** Modelagem das entidades de domínio de Acesso: Perfil com as seeds oficiais (**Admin, Staff, Moderador, Suporte, Player**), Permissao e PerfilPermissao.

• [ ] **3.3:** Implementação de persistência EF Core (mappings Fluent API, DbSet de Perfil, Permissão e PerfilPermissão e migrations).

• [ ] **3.4:** Desenvolvimento do CRUD completo com paginação e alteração de status para Usuario (GET, POST, PUT, PATCH /status, DELETE).

• [ ] **3.5:** Desenvolvimento dos endpoints CRUD paginados e gestão de status para Perfil e Permissao.

• [ ] **3.6:** Criação do SupportersController.cs com catálogo completo para o site (GET, GET admin, POST, PUT, PATCH /status, DELETE).

• [ ] **3.7:** Configuração de segurança RBAC por Policies no ApiConfig.cs, endpoints de autenticação corporativa e Web3 (/api/auth) e Swagger unificado.